Monte Carlo Methods

Botond Koroknai

March 15, 2026

Abstract

Monte Carlo methods are widely used numerical techniques for solving problems that involve uncertainty, high dimensionality, or complex domains where analytical solutions are difficult or impossible to obtain. This paper introduces the fundamental principles of Monte Carlo simulation and presents its mathematical formulation based on random sampling and statistical estimation. The theoretical convergence properties of the estimator are discussed, showing that the statistical error decreases proportionally to $1/\sqrt{N}$ as the number of samples increases.

To demonstrate the practical application of the method, a three–dimensional Monte Carlo integration example is implemented in C++. The program estimates the integral of the function $f(x, y, z) = e^{-x^2-y^2-z^2}$ over a spherical domain using randomly generated sample points. The implementation utilizes C++ features such as lambda expressions and the Mersenne Twister random number generator to provide a flexible and reusable framework for multidimensional integration. For validation, the Monte Carlo estimate is compared with a deterministic Riemann sum computed over a structured grid.

The results illustrate the efficiency of Monte Carlo integration for multidimensional problems, showing that reliable estimates can be obtained with significantly fewer function evaluations than traditional grid-based numerical methods. A visualization of the sampling process is also presented to provide an intuitive understanding of how random points contribute to the final integral estimate.

1 Monte Carlo Simulation

A Monte Carlo simulation is a computational technique based on repeated random sampling, used to model uncertainty and estimate the possible outcomes of a system or process.It can be utilised when a problem contains numerous uncertain variables or is too complex to solve analytically. [1].

In a Monte Carlo simulation, uncertain inputs are represented by probability distributions from which random samples are repeatedly drawn. Using these sampled inputs, the model produces a distribution of possible outcomes. As the number of simulations increases, the estimated results converge towards the true expected value due to the law of large numbers [2].

Monte Carlo simulations are widely used in fields such as finance, engineering, physics, and risk analysis to analyse complex systems in order to evaluate the impact of uncertainty on the outcomes of models. [2, 1].

2 Mathematical Formulation

The Monte Carlo simulation is based on approximating the expected value of a function using random sampling. Let X be a random variable with probability distribution $p(x)$ and let $f(X)$ be a function of interest. The expected value of $f(X)$ is defined as

$$E[f(X)] = \int f(x)p(x)\,dx.$$

In many practical problems this integral cannot be evaluated analytically, thus using the Monte Carlo methods, we can approximate the expected value by generating N independent random samples $x_1, x_2, \ldots, x_N$ from the distribution $p(x)$ and computing the sample mean

$$\hat{\mu} = \hat{E}[f(X)] = \frac{1}{N}\sum_{i=1}^{N} f(x_i).$$

Assuming the variance of $f(X)$ is σ^2, the variance of the estimator $\hat{\mu}$ is

$$\mathrm{Var}(\hat{\mu}) = \mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} f(x_i)\right).$$

Because the samples are independent, the variance of a sum equals the sum of the variances.

$$\mathrm{Var}(\hat{\mu}) = \frac{1}{N^2}\sum_{i=1}^{N}\mathrm{Var}(f(x_i)) = \frac{1}{N^2}N\sigma^2 = \frac{\sigma^2}{N}.$$

The standard deviation of the estimator, often referred to as the standard error, is therefore

$$\sqrt{\mathrm{Var}(\hat{\mu})} = \frac{\sigma}{\sqrt{N}}.$$

This result shows that the error of a Monte Carlo estimator decreases proportionally to $1/\sqrt{N}$, meaning that reducing the error by a factor of ten requires approximately one hundred times more simulations.

3 Practical Example: Monte Carlo Integration

The problem with numerical integration is that it evaluates the function on a fixed grid using fixed steps, which could become computationally expensive as the number of dimensions or the domain complexity increases.

On the other hand, Monte Carlo integration avoids this by randomly sampling points within a bounding cuboid and evaluating the integrand only at points inside the target domain. The final estimate is obtained by averaging these values and scaling by the cuboid's volume.

To understand how the Monte Carlo method works in practice, a simple demonstration of estimating a three-dimensional integral over a spherical region was created.

1. **Defining the Function and Domain:** Both the integrand and the integration domain are provided as lambda functions. The integrand lambda computes the value at a given point and the domain lambda returns true if a point is inside the region and false otherwise. This approach enables the Monte Carlo class to be flexible and reusable for any function or domain.

   Listing 1: Integrand and domain passed to Monte Carlo

   ```
   double result = MonteCarlo(
       [](double x, double y, double z)
       { return std::exp(-x*x - y*y - z*z); },
       [](double x, double y, double z)
       { return x*x + y*y + z*z < 16.0; },
       -4.0, 4.0, -4.0, 4.0, -4.0, 4.0);
   ```

   Here, the first lambda represents the function that we wish to integrate $f(x, y, z) = e^{-x^2-y^2-z^2}$. The second lambda defines the integration domain, which in this case is a sphere with a radius of 4.

2. **Random Sampling:** As a next step, the class generates random points using a Mersenne Twister engine and uniform distributions in each dimension:

   Listing 2: Random point generation

   ```
   std::random_device rd;
   std::mt19937 gen(rd());
   std::uniform_real_distribution<double> distX(xmin, xmax);
   std::uniform_real_distribution<double> distY(ymin, ymax);
   std::uniform_real_distribution<double> distZ(zmin, zmax);

   double x = distX(gen);
   double y = distY(gen);
   double z = distZ(gen);
   ```

   Each point is independent, which is crucial because the Monte Carlo estimate relies on unbiased sampling.

3. **Evaluating Points:** For each random point, the domain lambda checks if the point is inside the region. If it is, the integrand is evaluated and its value is accumulated:

Listing 3: Accumulating integrand values for points inside domain

```
if (domain(x, y, z)) {
    sum += integrand(x, y, z);
}
```

4. **Estimating the Integral:** After all samples are processed, the integral estimate is obtained by scaling the sum by the cuboid volume and normalizing by the number of samples:

Listing 4: Monte Carlo integral computation

```
double volume = (xmax - xmin) * (ymax - ymin) * (zmax - zmin)
    ;
double integral = sum / static_cast<double>(samples) * volume
    ;
```

5. **Control Value and Error Checking:** The class optionally computes a deterministic Riemann sum over the cuboid to serve as a control. In three dimensions, if the cuboid is divided into $n_x \times n_y \times n_z$ subintervals with step sizes $\Delta x, \Delta y, \Delta z$, the Riemann sum is given by:

$$I_{\text{Riemann}} = \sum_{i=0}^{n_x} \sum_{j=0}^{n_y} \sum_{k=0}^{n_z} f(x_i, y_j, z_k)\, \Delta x\, \Delta y\, \Delta z,$$

where (x_i, y_j, z_k) are the grid points inside the domain. The squared difference between the Monte Carlo estimate $\hat{I}_{\text{MC}}$ and this control value is then compared to a small tolerance ϵ to detect significant errors:

$$(\hat{I}_{\text{MC}} - I_{\text{Riemann}})^2 < \epsilon.$$

This ensures that the Monte Carlo implementation is producing a result consistent with a deterministic approximation.

In this implementation, the tolerance for the squared difference between the Monte Carlo estimate and the control Riemann sum was set to $\epsilon = 1 \times 10^{-4}$. The Monte Carlo integration used a standard sample size of 500,000 points, while the control integration employed 100 subdivisions along each axis, resulting in 1,000,000 evaluation points in total.

This comparison clearly demonstrates the advantage of the Monte Carlo method, as achieving the same level of precision in a high-dimensional or complex domain requires fewer evaluations than a traditional deterministic approach. In this example, the Monte Carlo estimator achieves reliable results with approximately half the number of function evaluations required by the deterministic control method, highlighting its efficiency and scalability for multidimensional integrals.

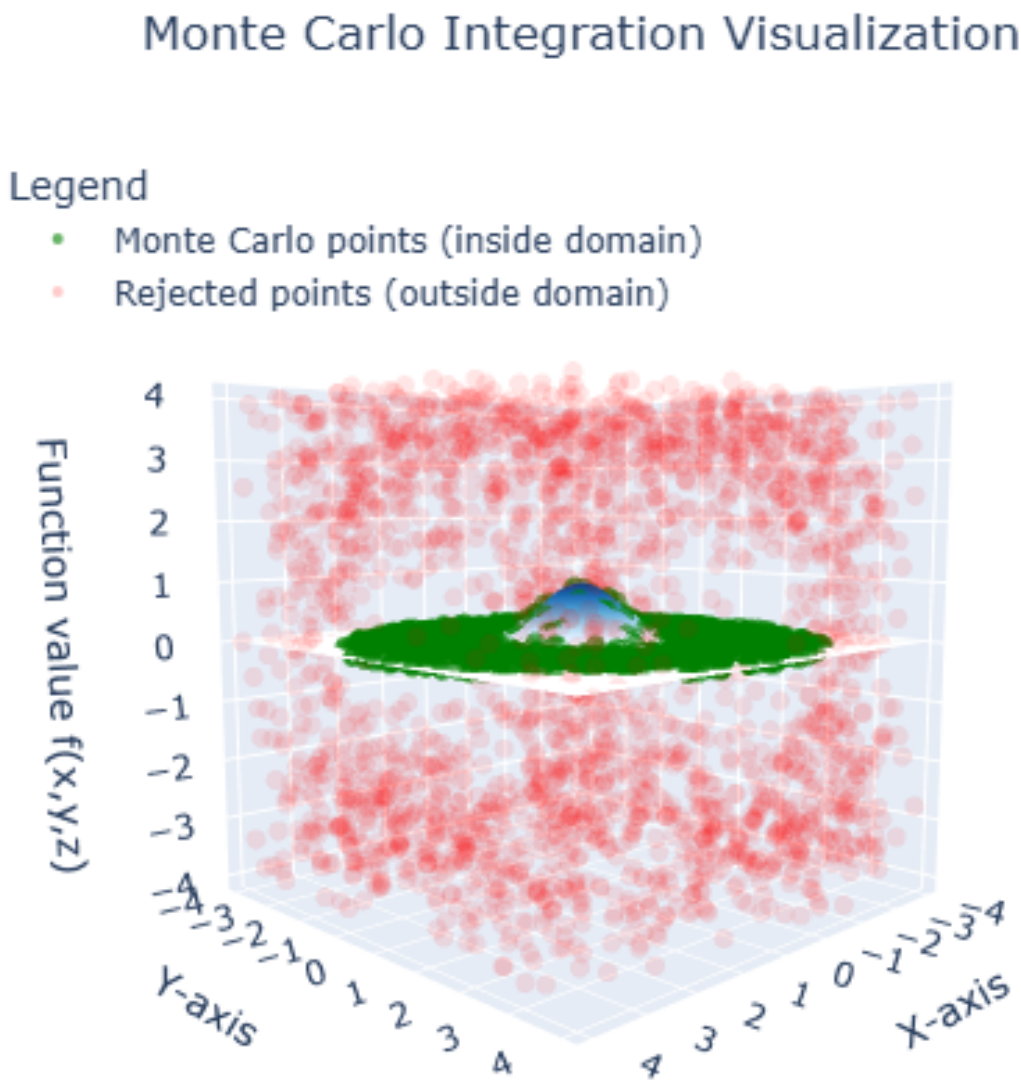

Figure 1: Monte Carlo sampling of $f(x, y, z) = e^{-x^2-y^2-z^2}$. Green points contribute to the integral; red points are rejected.

Figure 1 shows how Monte Carlo integration works. Random points are generated in a cube, and only points inside the spherical domain (green) contribute to the integral. Red points lie outside the domain and are ignored. The blue surface represents a 2D reference slice of the function, illustrating the exponential "bell" shape.

4 Conclusion

Monte Carlo methods provide a powerful numerical approach for approximating integrals and solving problems involving uncertainty or complex domains. By relying on random sampling rather than structured grids, these methods remain efficient even in higher-dimensional settings where traditional deterministic approaches become compu-

tationally expensive.

The presented implementation demonstrates how Monte Carlo integration can be applied to a three-dimensional problem and verifies the results through comparison with a deterministic Riemann sum. The visualization further illustrates how randomly sampled points contribute to the estimation of the integral, highlighting the intuitive and scalable nature of the Monte Carlo approach.

References

[1] George S. Fishman. *Monte Carlo: Concepts, Algorithms, and Applications*. Springer, 1996.

[2] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, 2004.